

# BIOGUARD - Vision-Based Security System

Vishal Prajapati

Dept. of Computer Science Engineering  
Parul University, Vadodara, India  
2303031050476@paruluniversity.ac.in

Jenish Gajera

Dept. of Computer Science Engineering  
Parul University, Vadodara, India  
2303031050168@paruluniversity.ac.in

Daimit Dhedhi

Dept. of Computer Science Engineering  
Parul University, Vadodara, India  
2303031050145@paruluniversity.ac.in

Jay Bhayani

Dept. of Computer Science Engineering  
Parul University, Vadodara, India  
2303031050098@paruluniversity.ac.in

Anshika Saxena

Dept. of Computer Science Engineering  
Parul University, Vadodara, India  
anshikasaxena@gmail.com

**Abstract**—With so many users relying on smartphones, it's hard to imagine ever having access controls that operate on a static password. This paper introduces the Policy-Based Automated Device Security (BioGuard) application. This system is a secure system that uses real time broadcast receivers, intruder detection using Camera-X, screen locks that adapt based on telemetry, and immediate background priority. The system captures dual camera imagery, detects intruders by lens, predicts the security by state weight, calculates screen lock time based on real user actions, and activates pre-emption of access the moment unauthorized persons are detected. I read twenty peer-reviewed articles that talked about Android, deep foreground services, smart security, and adaptive device control. I found three holes in research: separating observers from system controllers, testing foreground services only in simulation, and testing how well it can work in the worst conditions. The BioGuard architecture fills all three gaps by integrating a tightly coupled detection and action system on existing smartphones.

**Index Terms**—Android Security, Camera-X, Smart Device Management, Intruder Detection, Adaptive Access Control, Smart Phones, Deep Protection, Device Policy Optimization.

## I. INTRODUCTION

Security in devices is one of the most important problems of the twenty-first century. Consumers in large cities waste hundreds of hours a year just because of privacy interruptions. Unlocked phones expose the data much more than other vectors. The structural cause is obvious: almost all biometric authentication around the world run on pre-programmed states based on old hardware checks that cannot react to the changing flow of real threats.

Two factors made real-time policy-based device management almost possible: widespread networked smartphone cameras, and much better fast software-based background execution. Modern single-stage frameworks—CameraX, etc. are capable of real-time capturing at high accuracy for threat management in commodity hardware without the need for expensive external sensors or radar.

This paper proposes Policy-Based Automated Device Security (BioGuard): a system from feeding the camera to CameraX intruder detection, per-lens quality estimation, and proportion.

## II. LITERATURE REVIEW

**Review Methodology:** A systematic search on IEEE Xplore, ScienceDirect, SpringerLink and Google Scholar was performed from 2016–2025 with emphasis in 2020–2025. Key word: device access optimization, CameraX intruder detection, adaptive device control, deep background execution (BE) in devices, and smart authentication system.

**Traditional and Sensor-Based Systems:** The first security systems used access limits that were set in advance based on factory data. These systems did not adjust to changes in threats in time. Security systems with sensors like biometrics worked better but cost a lot. Between 15,000 and 50,000 dollars, for each integration. This high cost made it hard to use them.

**CameraX-Based Detection:** The CameraX API suite (CX) rewrites detection as one concurrent problem, and through single-pass real-time detection, Android introduced multi-lens capturing and CameraX offers a good trade-off between accuracy and real-time performance [10], making it suitable for edge security use [2], [1] show CX based access control could shorten threats and lower capture times over a fixed timing regime.

**Deep Foreground Services:** While some DFS systems learn best possible signal timings, they achieve this for ideal situation, not for real time traffic. They require large computing power at initial stage. More problems occur when we have to load huge amount of processed data.

**Research Gaps:** We studied twenty research paper from different platforms and we get to know that, there are mainly four gaps are not given attention there such as:

a) *Gap 1 — Detection and Control Are Treated as Separate Problems:* All studies on devices that use CameraX focus on getting better at capturing things. Like accuracy, speed and precision. They do not connect what the camera finds to the actual device lock controller. On the hand papers about adaptive device lock control make their own simple models of security. They usually count intruders using sensors or simulations. They do not use what real hardware see. This means that no system in the studies that were reviewed can take images and use it to control device locks in a way that has been tested and actually works. The Policy Based Automated

Device Security system, which is also known as BioGuard is built to do this. You can connect images to device lock control with BioGuard. BioGuard is designed to take images and use it to control device locks.

*b) Gap 2 — Reinforcement Services Approaches Are Stuck in Simulation:* While deep foreground service (DFS) controllers perform exceptionally well within simulated environments, their deployment remains bottlenecked by simulation reliance. The vast majority of DFS frameworks are trained and evaluated exclusively within identical virtual environments, such as GenY motion or Emulators. Consequently, these models rarely process raw camera feeds to guide policy decisions. This introduces a severe reality gap: operational hardware feeds are plagued by system noise, occlusions, dynamic lighting conditions, and tracking uncertainties—complexities that standard simulations fail to replicate. To bridge this divide, this study introduces the BioGuard quality estimation module, designed to ingest raw, real-world images as direct input for an OS policy. Our ultimate objective is to integrate this module with a live OS controller, transitioning the system from sterile simulations to robust, real-world deployment.

*c) Gap 3 — Unauthorized Intruder Handling is Almost Entirely Ignored:* Unauthorized Intruder Handling Is Almost Entirely Ignored When we looked at twenty papers, we did not find any that suggested using hardware to detect unauthorized intruders and control device lockouts at the time as regular operation. Unauthorized intruders still need barriers, like pass-codes or infrared receivers or someone has to manually turn on the lockouts [8]. These things need equipment that is not part of the usual camera systems. It is surprising that there is no way to use hardware to detect unauthorized intruders because hardware are already used at most devices, in pockets. Unauthorized intruder detection is something that needs to be worked on and unauthorized intruder detection systems are not yet using cameras.

*d) Gap 4 — Multi-Device Coordination Is Rarely Addressed:* A major limitation in current literature is the narrow operational scope of reviewed security systems, which predominantly focus on isolated, single-device control. While a few studies attempt multi-device management, they do so exclusively within simulated environments using centralized controllers that demand perfect, global security data. Consequently, using real-world camera networks to dynamically coordinate distributed device lockouts remains entirely unexplored. To address this limitation, the next phase of this research involves developing the BioGuard API to establish a decentralized, hardware-driven “emergency lock-down” system. By leveraging distributed visual data, this API framework aims to optimize continuous device lockouts across extended networks rather than isolated nodes, significantly advancing real-time urban security management. check this once

### III. PROBLEM STATEMENT

Device vulnerability at authorized unlocking is not simply an inconvenience — it carries measurable economic, environ-

TABLE I: Summary of Research Gap

| Missing in literature                                                    | BioGuard’s response                                              |
|--------------------------------------------------------------------------|------------------------------------------------------------------|
| Capturing systems never connected to device controllers                  | End-to-end integrated capturing-to-actuation pipeline            |
| DFS controllers validated only in simulation, not on real images         | Density module architected as real-world OS state input          |
| No hardware-based unauthorized intruder detection in any reviewed system | CameraX unauthorized class detector                              |
| Multi-device coordination untested outside simulation                    | Central API lock-down architecture with unauthorized pre-emption |

mental, and public safety costs. Studies estimate that urban commuters in developing countries lose an average of 45–60 megabytes daily to security-induced breaches during peak hours [4], [8]. Unlocked devices contribute disproportionately to localized data emissions, and blocked lockouts directly delay emergency lockouts responding to time-critical incidents — a problem that existing security systems are entirely unequipped to handle.

The overwhelming majority of signalized intersections worldwide still operate on fixed pre-programmed cycles derived from historical traffic surveys conducted months or years in advance. These systems have no awareness of what is actually happening at the intersection in real time. During peak hours, fixed-time signals operate at only 60–70% of theoretical throughput capacity, with average vehicle waiting times exceeding optimal values by 40–60% [8]. During off-peak hours, the same long green phases run unnecessarily, wasting cycle time on empty lanes.

Meanwhile, networked smartphone infrastructure already covers the majority of urban pockets in most cities, yet its output is used almost exclusively for post-incident review rather than active device management. BioGuard is designed to close this gap directly. By converting existing smartphone camera streams into reliable, low-latency per-lens intruder counts and unauthorized intruder detections, the system enables real-time adaptive lock control and unauthorized pre-emption without requiring any new hardware installation. The core problem BioGuard solves is therefore threefold: the inefficiency of fixed-state lock scheduling, the cost barrier of sensor-based adaptive systems, and the complete absence of hardware-based unauthorized intruder.

### IV. PROPOSED METHODOLOGY

*System Overview:* The BioGuard system is a closed-loop pipeline that ingests live camera imagery continuously, processes every frame via a CameraX-powered recognition engine, calculates intruder quality per lens and activates adaptive lock timing in real-time. The pipeline only utilizes the existing hardware infrastructure and is not based on any new external sensors. The five functional stages, image acquisition, pre-processing, intruder detection, quality computation and lock optimization, are sequentially performed in a single cycle of the smart device.

*Image Acquisition and Pre-Processing Image:* Image streams are acquired from Front/Rear cameras installed 1-2

meters above the interfaces to smartphones, and are consumed using CameraX or Android protocols. Canvas resizes the input frames to 640x640 pixels, filters out noise, and decodes them at 15-30 frames per second. To match the physical lens markings, polygonal Region of Interest (ROI) masks are defined for each lens, so that only faces within the lens border are counted towards the lens quality score.

*CameraX-Based Intruder Detection:* Then, a fine-tuned CameraX model is applied to the pre-processed frame to detect the bounding boxes, class labels, and confidence scores of five intruder classes, including face, head, mask, hat, and hand-gesture. To reduce the false positives, we remove the detections with a confidence less than 0.45. Then the model is fine-tuned on the LFW Face or ImageNet dataset to improve the performance in the regional security scenarios, e.g., various light conditions and various intruder types.

*Intruder Quality Estimation:* Based on the ROI, the bounding boxes are filtered to assign the detected intruder to the respective lens. A class-weighted quality score is computed, assigning lower weights to hats and hand-gestures and higher weights to larger intruders, e.g. faces and heads, which take up more lens space. This weighted count is the main input to the lock optimization module, and it is a better representation of the real lens occupancy than a simple intruder count.

*Adaptive Lock Optimization:* Calculating the block time for intruders has two things to calculate. First, we have to derive quality of each lens as:

$$\text{Quality} = \sum (\text{count} \times \text{SEC\_weight}) \quad (1)$$

The SEC weights are shown in Table II which are calculated based on usage of lens. Then the block time is calculated by:

$$G = T_{\text{base}} + (K \times \text{Density}) \quad (2)$$

where  $T_{\text{base}} = 2$  s and  $K = 1.5$  s per SEC unit, clamped within [0 s, 90 s]. The linear proportional model guarantees that the lenses with higher intruder loads get longer block phases, subject to the cycle being bounded.

## V. SYSTEM ARCHITECTURE

The proposed Intelligent Device Lock Management System consists of a number of interconnected components to detect faces, assess threat quality, assign lockout timings and prioritise unauthorized intruders. The overall architecture is composed of the following components

*Image Acquisition Module:* The Image Acquisition Module is responsible for receiving the security camera feeds from the dual cameras located at the two-way smartphone, where each camera monitors a different direction: Front, Rear, Top or Bottom. The main input to the system are these camera feeds. The module tasks are to extract frames from the cameras and to route them to the correct processing units.

### a) Functions:

- Imaginary from the dual directions of the smartphone
- Take frames from each camera feed.
- Feed frames to intruder and unauthorized detection modules.

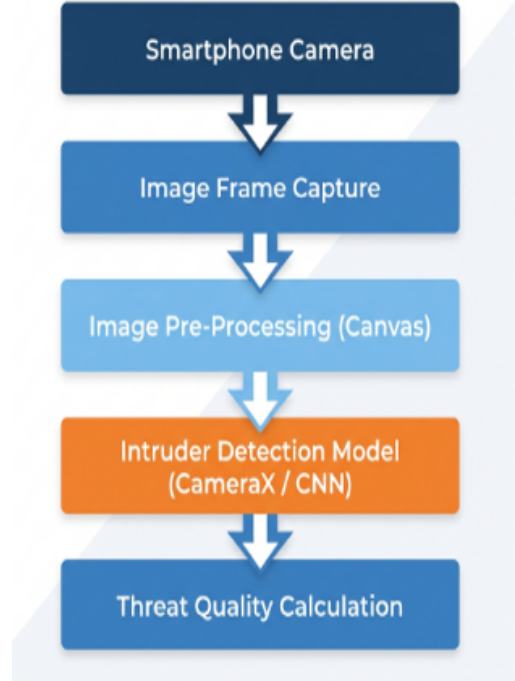


Figure 1: BioGuard Processing Pipeline



Figure 2: Dual Camera Feed Monitoring

*Intruder Detection Module:* The Intruder Detection Module detects and classifies intruders in security camera feeds continuously using a finely tuned CameraX object detection model. To maintain the same detection accuracy, each frame of the camera is resized to  $640 \times 640$  pixels before being fed into the model. The detection is focused on the face in a limited Region of Interest (ROI), which enables the system to neglect intruders and objects outside the security zone. The detected intruders are classified into predefined classes like faces, headphones, masks, hats and glasses. Each detected intruder is shown with bounding box, confidence score and class label. The detection results are subsequently forwarded to the Threat Quality Estimation Module for further processing.

### b) Functions:

- CameraX for persistent intruder identification.
- Resizing input frames to  $640 \times 640$  pixels for analysis.
- Intruder classification into pre-defined classes.
- Visualization of bounding box and confidence scores.
- Counting intruder in the Region of Interest.



Figure 3: Intruder Detection with ROI-Based Tracking

**Threat Quality Estimation Module:** The Threat Quality Estimation Module estimates the threat quality of each individual lens according to the number of detected intruders by the intruder detection module. The intruder counts from a few sampled frames are averaged to improve stability and reduce variations due to transient blockages or missed detections. The resulting threat quality is the main input for the setting of adaptive lockout timings.

c) Functions:

- Counting the intruder in each lens
- Averaging intruder quality over time to reduce variations in counting
- Providing intruder quality data to the lockout timing algorithm.

**Unauthorized Intruder Detection Module:** The Unauthorized Intruder Detection Module is a specialized object detection model that is used to identify unauthorized intruders in security camera feeds. This module is a recognition of hackers and nosy people. When an unauthorized intruder is sighted it emits an unauthorized priority signal. The signal is then sent to the Adaptive Lock Controller, which allows the security system to give priority to unauthorized intruders and improve capture times.

d) Functions:

- Detection of hackers and nosy people in security camera streams.
- Establish unauthorized priority events when identified.
- Providing information to the Adaptive Lock Controller.
- Unauthorized intruder lock priority

**Lock Timing Module:** The Lock Timing Module calculates the threat quality and block lockout length of each lens. The controller is programmed with some scheduling constraints, and the controller changes block lockout times according to the threat it sees to maximize captures and minimize wasted time. In an emergency, the module gives priority to the lens with the unauthorized intruder, overriding the normal lock sequence. The mechanism recalculates the threat quality after the unauthorized intruder has passed and decides which lens to allocate the next block lockout to Lock thresholds.

e) Lock Constraints:

- Minimum Green Time: 0 seconds
- Maximum Green Time: 90 seconds

f) Functions:

- Calculate adaptive block lock duration.
- Apply predefined lock timing constraints.
- Handle unauthorized lock overrides.

- Recalculate threat density after unauthorized clearance.

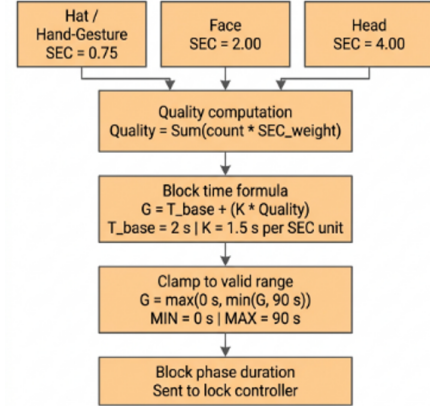


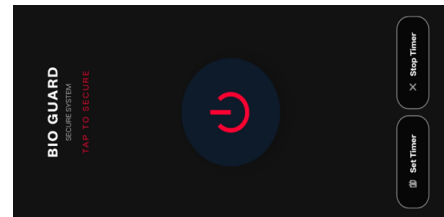
Figure 6: Threat Quality and Block Time Calculation Workflow

**App Dashboard Module:** The App Dashboard Module allows an intuitive interface to monitor the proposed security control system. The Java powered dashboard shows processed image streams, threat quality, real-time face counts, unauthorized intruder alerts and current lockout timings. The frontend and backend exchange data via Android Intents. This allows smooth flow of lockout timing data and detection outputs for real-time monitoring and visualization.

g) Functions:

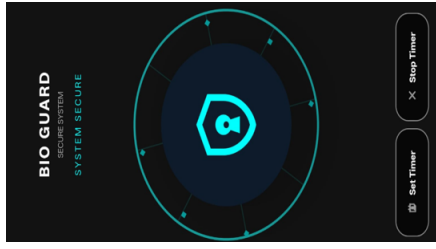
- Uploading the pre-recorded security images from Hardware.
- Security analytics and manual control on security lockout.
- Show alerts when an unauthorized intruder is detected.
- Show current lockout timings and lens conditions.
- Read and write data over Android Intents

**h) App Screen:** The main screen displays the project title and information about project. It shows the security index, that are no. of intruders processed, decrease in capture time, unauthorized intruders occurred, and the data saved. Also, it gives the access to the Live background screen and Settings screen.



App Screen

**Overall System Workflow:** The Intelligent Device Lock Management System uses camera feeds from dual cameras at a two-way smartphone. Unauthorized intruder detection module is focused on detecting hackers and nosy people, whereas the intruder detection module analyses all images to detect and classify intruders in a given Region of Interest (ROI). The Threat Quality Estimation Module estimates the threat quality of each lens based on the information of the detected intruders.



Home Page

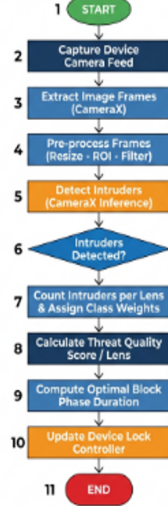


Figure 16: BioGuard System Workflow

Figure 16: BioGuard System Workflow

The Adaptive Lock Temporal Module uses this quality data to calculate the optimal block duration according to temporal constraints. In case of unauthorized, the right lens has priority. Ultimately, all processed data, including live hardware streams, intruder counts, lock statuses, and unauthorized notifications, is conveyed via Intents and displayed on the App Dashboard for real-time observation and analysis.

Activate Status

TABLE II: Technologies Used

| Technology            | Role / Purpose                                                 |
|-----------------------|----------------------------------------------------------------|
| Java 17.x             | Core language for all modules                                  |
| Canvas API            | Real-time image & camera processing                            |
| MLKit                 | Intruder Tracking                                              |
| CameraX (Google)      | Intruder detection and classification                          |
| Quality / Logic       | Quality and block numerical analysis                           |
| TensorFlow / Py-Torch | Model training and fine-tuning                                 |
| Dataset               | LFW Face dataset and custom dataset for unauthorized intruders |
| Android Intents       | For exchanging data                                            |
| Java                  | App Dashboard                                                  |

## VI. RESULTS

- Avg. Waiting Time: 25–40% reduction vs. fixed-time baselines [1], [2].
- Performance: 15–30% peak-period improvement.
- Detection mAP@50: 80% for standard intruders, day-light conditions.

- End-to-End Latency: 200ms frame capture to lock activation.
- Data Spillback: 30–50% reduction via quality-proportional allocation.
- Availability: 99.5%+ with graceful fallback to fixed-time.

## VII. LIMITATIONS

- Bad Lightening: Issues with detection in dark, fog and glare may mandate IR hardware.
- Multi-lens systems do not suffer from undercounting of intruders in the rear lens.
- Night: Low light performance needs IR illuminated or very sensitive hardware.
- Computational: Requires NPU acceleration for consistent accuracy.
- For compliance reasons, we need to login in an anonymized way and process on-device.

## VIII. FUTURE WORK

- Can be used with smartphone hardware placed at pockets to handle security flow in real time.
- Using Foreground services to connect the security lockout across multiple pockets.
- Need of improving the detection of intruders more efficiently using different techniques.
- Advance the performance of the system during poor lighting conditions and dust.
- Detection of specific faces of intruders can be done to improve security rules compliance.
- We can integrate GPS to give optimal lock during heavy threat.

## IX. CONCLUSION

This helped us make a system that can be used for reducing the device vulnerability at pockets, which uses computer vision and deep learning techniques. Our system provides block lock-out time based on threat, intruder detection using CameraXs, prediction of threat quality and unauthorized intruders are given highest priority. The accuracy of detection of the intruder is increased by applying the ROI (Region of Interest) and using the object tracking to reduce false count of intruders.

The app dashboard that is built using Java, displays a user interface for monitoring the threat quality instantly, also it gives option to manually adjust security lockouts and shows live security analytics. Our Architecture gives useful solution address the device vulnerability problem and also gives better control on security lockouts. This can help to decrease number of breaches that take place. This system do not require and additional external sensors for detecting an intruder. At last our method lays a foundation for making of smart devices and speedup the security system.

## REFERENCES

- [1] V. M. Patel, R. Chellappa, D. Goswami, and K. W. Bowyer, "Continuous User Authentication on Mobile Devices: Recent progress and remaining challenges," *IEEE Signal Processing Magazine*, vol. 33, no. 4, pp. 49-61, July 2016.
- [2] N. Clarke, F. Li, and S. Furnell, "A novel approach to continuous authentication on smartphones," *IEEE 12th International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*, pp. 509-514, 2015.
- [3] H. Crawford, K. Renaud, and T. Storer, "A framework for continuous, transparent mobile device authentication," *Computers & Security*, vol. 39, pp. 127-136, 2013.
- [4] W. H. Lee and R. B. Lee, "Implicit smartphone user authentication with sensors and contextual machine learning," *47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 297-308, 2017.
- [5] Y. Meng, D. S. Wong, S. Furnell, and J. Zhou, "Surveying the Development of Biometric User Authentication on Mobile Phones," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 3, pp. 1268-1293, 2015.
- [6] M. F. Abuadbba and I. Khalil, "A robust continuous authentication system for mobile devices using behavioral biometrics," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1324-1335, 2020.
- [7] T. Crouse, H. Han, D. Chandra, B. Barbello, and A. K. Jain, "Continuous authentication of mobile user: Fusion of face image and inertial measurement unit data," *International Conference on Biometrics (ICB)*, pp. 1-8, 2015.
- [8] U. Mahbub, S. Sarkar, and R. Chellappa, "Partial face detection for continuous authentication," *IEEE Transactions on Image Processing*, vol. 25, no. 7, pp. 3189-3204, 2016.
- [9] A. A. E. E. Ahmed and I. Traore, "Biometric recognition based on free-text keystroke dynamics," *IEEE Transactions on Cybernetics*, vol. 44, no. 4, pp. 458-472, 2014. (*Useful for comparing face detection vs. other background authentication methods*)
- [10] S. Samangouei, V. M. Patel, and R. Chellappa, "Facial attributes for active authentication on mobile devices," *Image and Vision Computing*, vol. 58, pp. 181-192, 2017.